College of Engineering and Applied Sciences

Course Name: Lab. Intro. To Microprocessors

Course Number: ECE 2510

Instructors:
Christian Blanchard
Pravana Avuthu

Lab #5

Student:

De Leon Santos, Sebastian Manuel – WIN: 577866168

Task 1:

- Assembly Code

C:\Users\sebas\wmu_classes\spring2026\intro_microprocessors\lab\lab5\task1\task1.uvprojx - µVision [Non-Commercial Use License]

File Edit View Project Flash Debug Peripherals Tools SVCS Window Help

Target_1

Project

- Project: task1
  - Target_1
    - Source Group 1
      - main.s
    - CMSIS

main.s

```
;ARM assembly code shell

        PRESERVE8
        THUMB

; Vector Table
        AREA    RESET, DATA, READONLY
        EXPORT  __Vectors

__Vectors
        DCD     0x20018000          ; Top of Stack (96KB RA
        DCD     Reset_Handler       ; Reset Handler

        AREA    |.text|, CODE, READONLY

; Reset Handler
Reset_Handler   PROC
                EXPORT  Reset_Handler
                BL      main
                B       .
                ENDP


main            PROC
                EXPORT  main

                ldr r12, =0x20000000
                ldr r11, =0x0           ; r11 will hold car

                ; first byte
                ; 72 + 25
                ; adding first byte
                ldr r0, =0x72
                ldr r1, =0x25

                and r2, r0, #0x0F
                and r3, r1, #0x0F

                and r4, r0, #0xF0
                and r5, r1, #0xF0
```

C:\Users\sebas\wmu_classes\spring2026\intro_microprocessors\lab\lab5\task1\task1.uvprojx - µVision [Non-Commercial Use License]

File Edit View Project Flash Debug Peripherals Tools SVCS Window Help

Target_1

Project

- Project: task1
  - Target_1
    - Source Group 1
      - main.s
    - CMSIS

main.s

```
                and r3, r1, #0x0F

                and r4, r0, #0xF0
                and r5, r1, #0xF0

                ; add first digit
                adds r6, r2, r3
                cmp r11, #1          ; check carry flag
                addeq r6, #1
                moveq r11, #0
                cmp r6, #0x09        ; compare if greater th
                movgt r11, #1

                cmp r11, #1          ; check carry flag
                addeq r7, #1
                moveq r11, #0
                adds r7, r4, r5
                cmp r7, #0x90
                movgt r11, #1

                add r8, r7, r6
                strb r8, [r12, #2]   ; store the byte

                ldr r0, =0x42
                ldr r1, =0x89

                and r2, r0, #0x0F
                and r3, r1, #0x0F

                and r4, r0, #0xF0
                and r5, r1, #0xF0

                ; add first digit
                adds r6, r2, r3
                cmp r11, #1          ; check carry flag
                addeq r6, #1
                moveq r11, #0
                cmp r6, #0x09        ; compare if greater th
                movgt r11, #1
                addgt r6, #0x06
```

Simulation

C:\Users\sebas\wmu_classes\spring2026\intro_microprocessors\lab\lab5\task1\task1.uvprojx - µVision [Non-Commercial Use License]

File Edit View Project Flash Debug Peripherals Tools SVCS Window Help

Target_1

Project: task1
- Target_1
  - Source Group 1
    - main.s
  - CMSIS

main.s

```
                addgt r6, #0x06

                cmp r11, #1          ; check carry flag
                addeq r7, #1
                cmp r11, #1
                moveq r11, #0
                adds r7, r4, r5
                cmp r7, #0x90
                movgt r11, #1
                addgt r7, #0x60

                add r8, r7, r6
                strb r8, [r12, #1]   ; store the byte


                ldr r0, =0x69
                ldr r1, =0x37

                and r2, r0, #0x0F
                and r3, r1, #0x0F

                and r4, r0, #0xF0
                and r5, r1, #0xF0

                ; add first digit
                adds r6, r2, r3
                cmp r11, #1          ; check carry flag
                addeq r6, #1
                cmp r11, #1
                moveq r11, #0
                cmp r6, #0x09        ; compare if greater th
                movgt r11, #1
                addgt r6, #0x06

                cmp r11, #1          ; check carry flag
                addeq r7, #1
                cmp r11, #1
                moveq r11, #0
                adds r7, r4, r5
                cmp r7, #0x90
                movgt r11, #1
```

Simulation

C:\Users\sebas\wmu_classes\spring2026\intro_microprocessors\lab\lab5\task1\task1.uvprojx - µVision [Non-Commercial Use License]

File Edit View Project Flash Debug Peripherals Tools SVCS Window Help

Target_1

Project

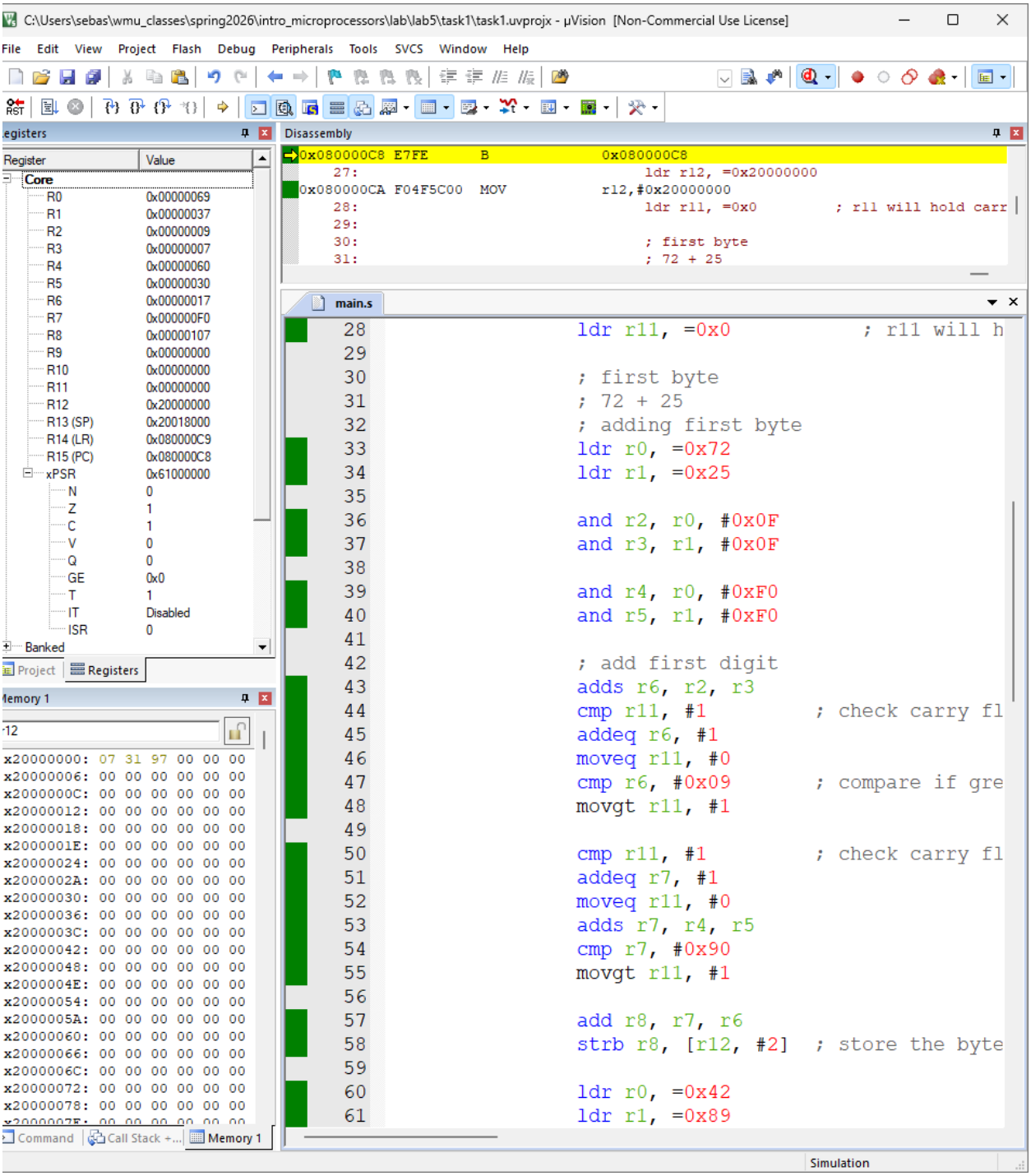
- Project: task1
  - Target_1
    - Source Group 1
      - main.s
    - CMSIS

main.s

```
                strb r8, [r12, #1]  ; store the byte


                ldr r0, =0x69
                ldr r1, =0x37

                and r2, r0, #0x0F
                and r3, r1, #0x0F

                and r4, r0, #0xF0
                and r5, r1, #0xF0

                ; add first digit
                adds r6, r2, r3
                cmp r11, #1         ; check carry flag
                addeq r6, #1
                cmp r11, #1
                moveq r11, #0
                cmp r6, #0x09       ; compare if greater th
                movgt r11, #1
                addgt r6, #0x06

                cmp r11, #1         ; check carry flag
                addeq r7, #1
                cmp r11, #1
                moveq r11, #0
                adds r7, r4, r5
                cmp r7, #0x90
                movgt r11, #1
                addge r7, #0x60

                add r8, r7, r6
                strb r8, [r12, #0]  ; store the byte


                BX      LR              ; Return from main
                ENDP

                ALIGN
                END
```

Simulation

• Screenshot showing the results.



Result stored in 0x20000000: 073197

Task 2:

• For each branch instruction, calculate the branch offset.

Program starts at: back B forward: 0x080000CA

Goes to forward NOP: 0x080000D2

Runs: B back: 0x080000D4

Goes back to: back B forward: 0x080000CA

And continue looping jumping to those addresses.

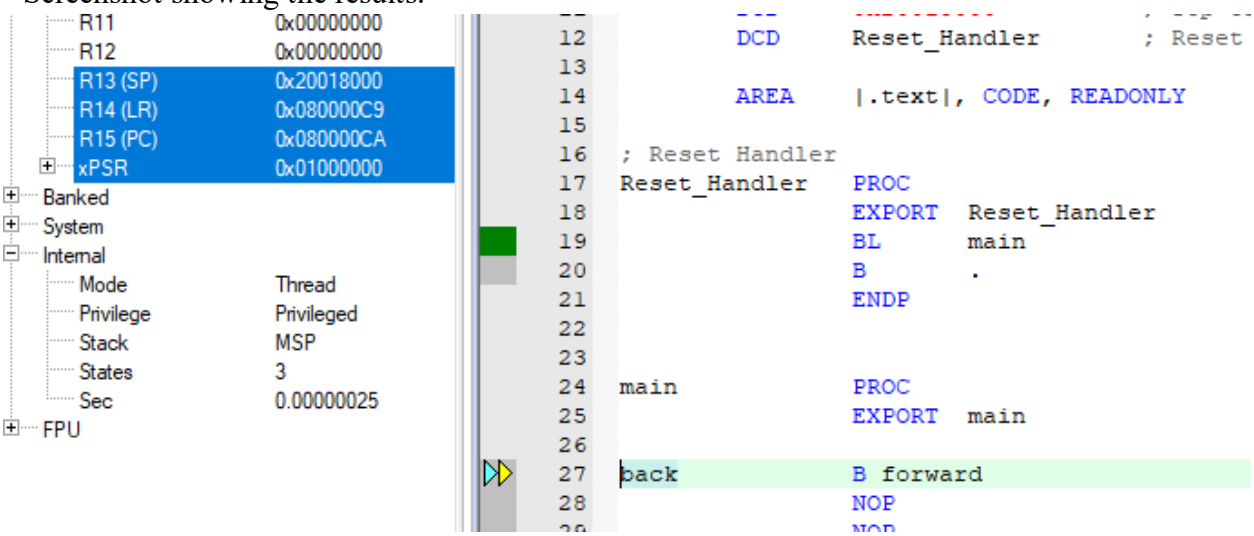
The programs branch to forward executes the NOP instruction which does nothing (No operation), then, branches to back, then, branches to forward without executing the NOPs below the back label and loops indefinitely.

Branch offsets:

B forward: target – current => 0x080000D2 – 0x080000CA = 8

B back: target – current => 0x080000CA – 0x080000D4 = 0xFFFFFFF6

• Screenshot showing the results.



R11	0x00000000
R12	0x00000000
R13 (SP)	0x20018000
R14 (LR)	0x080000C9
R15 (PC)	0x080000D2
xPSR	0x01000000
Banked	
System	
Internal	
Mode	Thread
Privilege	Privileged
Stack	MSP
States	6
Sec	0.00000050
FPU	

```
12          DCD     Reset_Handler        ; Reset
13
14          AREA    |.text|, CODE, READONLY
15
16  ; Reset Handler
17  Reset_Handler   PROC
18                  EXPORT  Reset_Handler
19                  BL      main
20                  B       .
21                  ENDP
22
23
24  main            PROC
25                  EXPORT  main
26
27  back            B forward
28                  NOP
29                  NOP
30                  NOP
31
32  forward         NOP
33                  B back
34
35                  BX      LR
```

R9	0x00000000
R10	0x00000000
R11	0x00000000
R12	0x00000000
R13 (SP)	0x20018000
R14 (LR)	0x080000C9
R15 (PC)	0x080000D4
xPSR	0x01000000
Banked	
System	
Internal	
Mode	Thread
Privilege	Privileged
Stack	MSP
States	7
Sec	0.00000058
FPU	

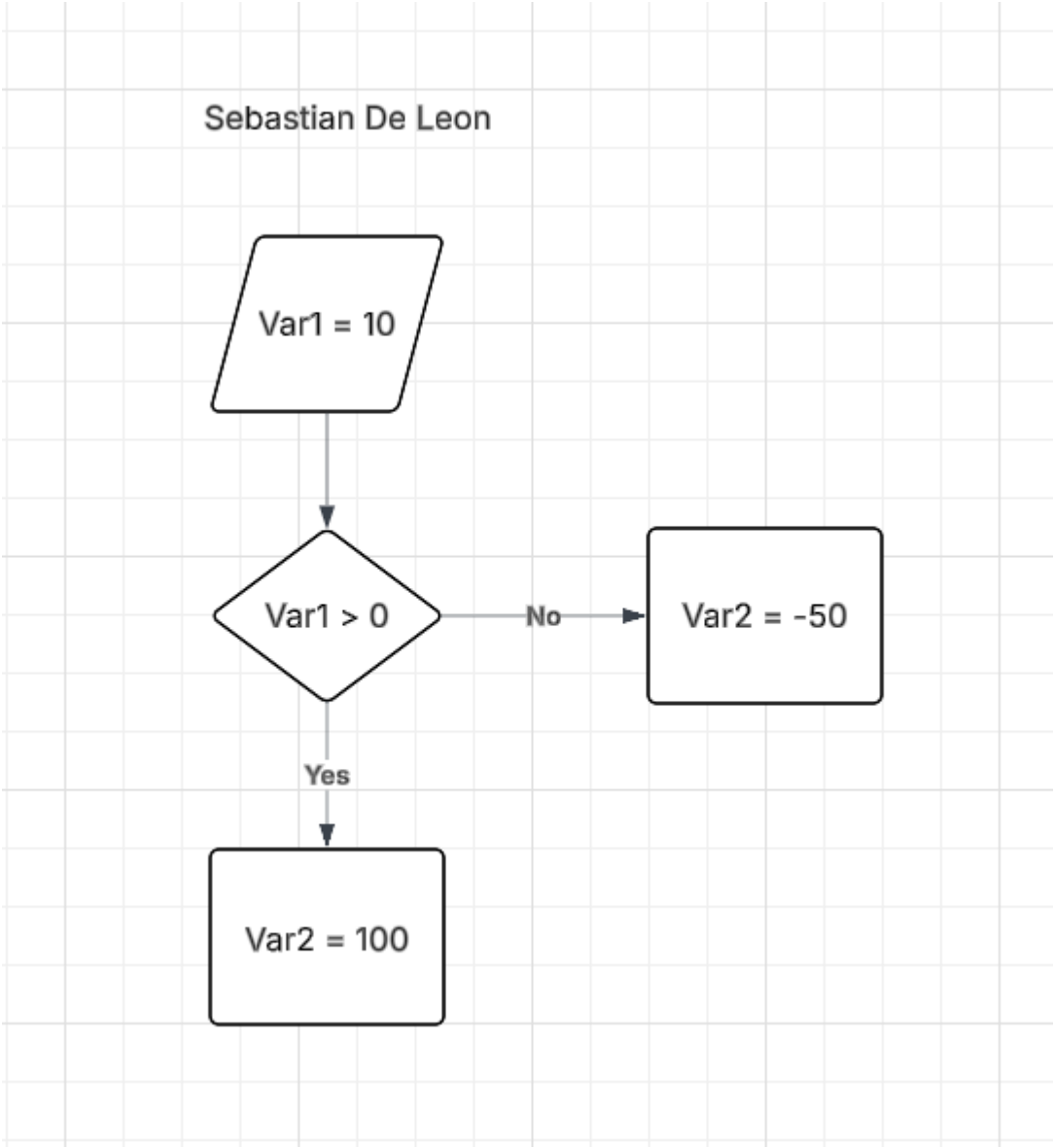
```
10  __Vectors
11          DCD     0x20018000           ; T
12          DCD     Reset_Handler        ; R
13
14          AREA    |.text|, CODE, READONLY
15
16  ; Reset Handler
17  Reset_Handler   PROC
18                  EXPORT  Reset_Handler
19                  BL      main
20                  B       .
21                  ENDP
22
23
24  main            PROC
25                  EXPORT  main
26
27  back            B forward
28                  NOP
29                  NOP
30                  NOP
31
32  forward         NOP
33                  B back
34
35                  BX      LR
```

R8	0x00000000
R9	0x00000000
R10	0x00000000
R11	0x00000000
R12	0x00000000
R13 (SP)	0x20018000
R14 (LR)	0x080000C9
R15 (PC)	0x080000CA
xPSR	0x01000000
Banked	
System	
Internal	
Mode	Thread
Privilege	Privileged
Stack	MSP
States	10
Sec	0.00000083
FPU	

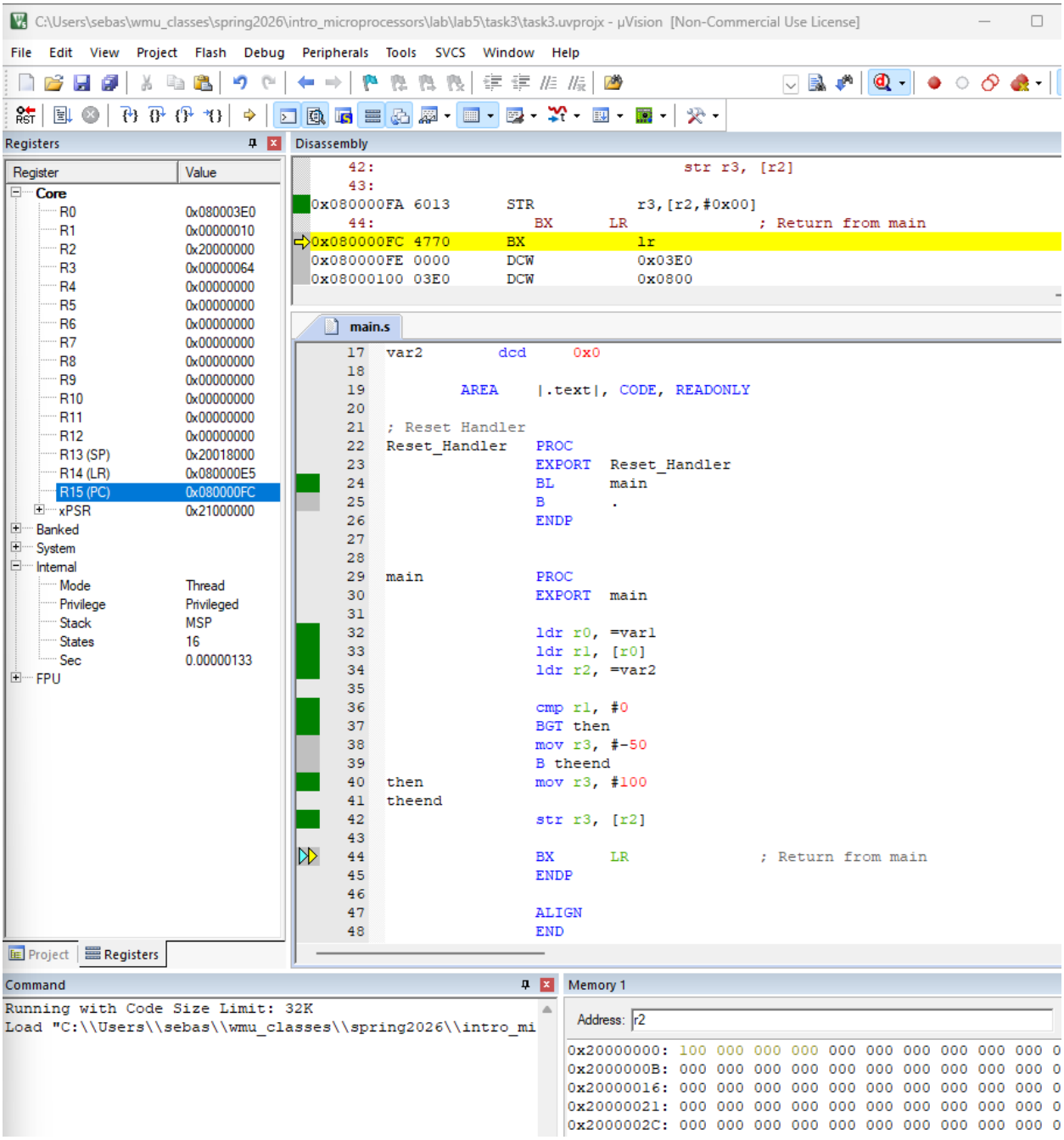
```
__Vectors
        DCD     0x20018000          ; Top of Stack
        DCD     Reset_Handler       ; Reset Handle

        AREA    |.text|, CODE, READONLY

; Reset Handler
Reset_Handler   PROC
                EXPORT  Reset_Handler
                BL      main
                B       .
                ENDP


main            PROC
                EXPORT  main

back            B forward
                NOP
                NOP
                NOP
```

Task 3:

• First, draw a flowchart representing the program.

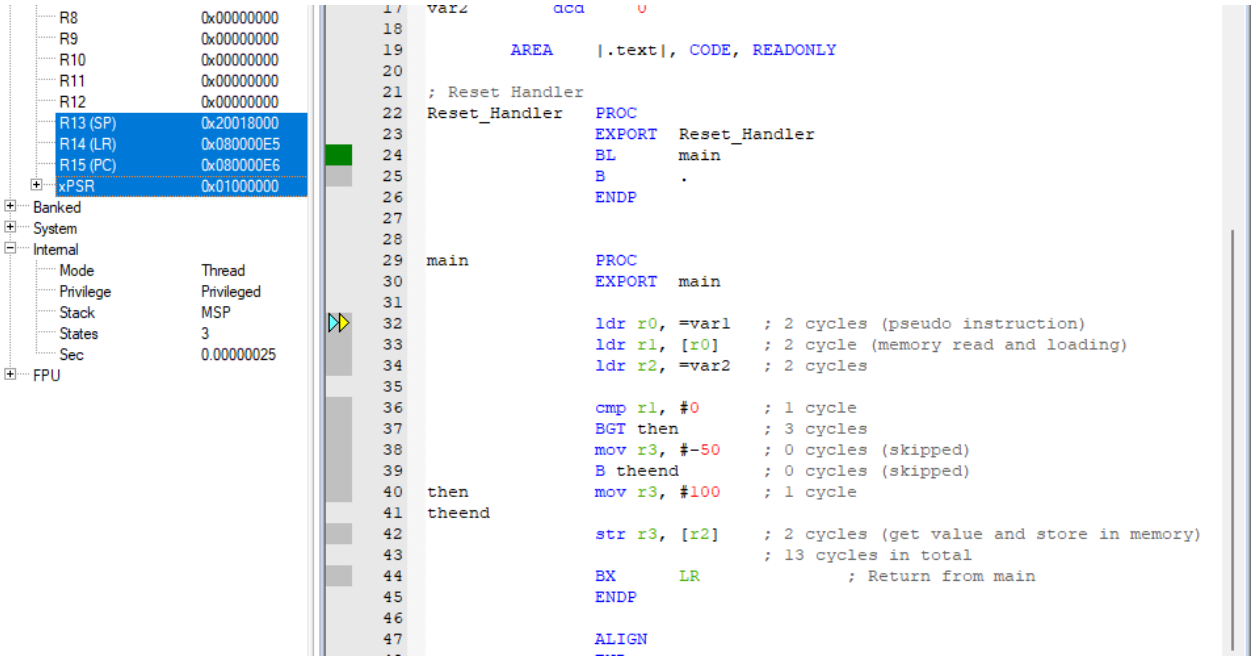


• Write the ARM assembly program.

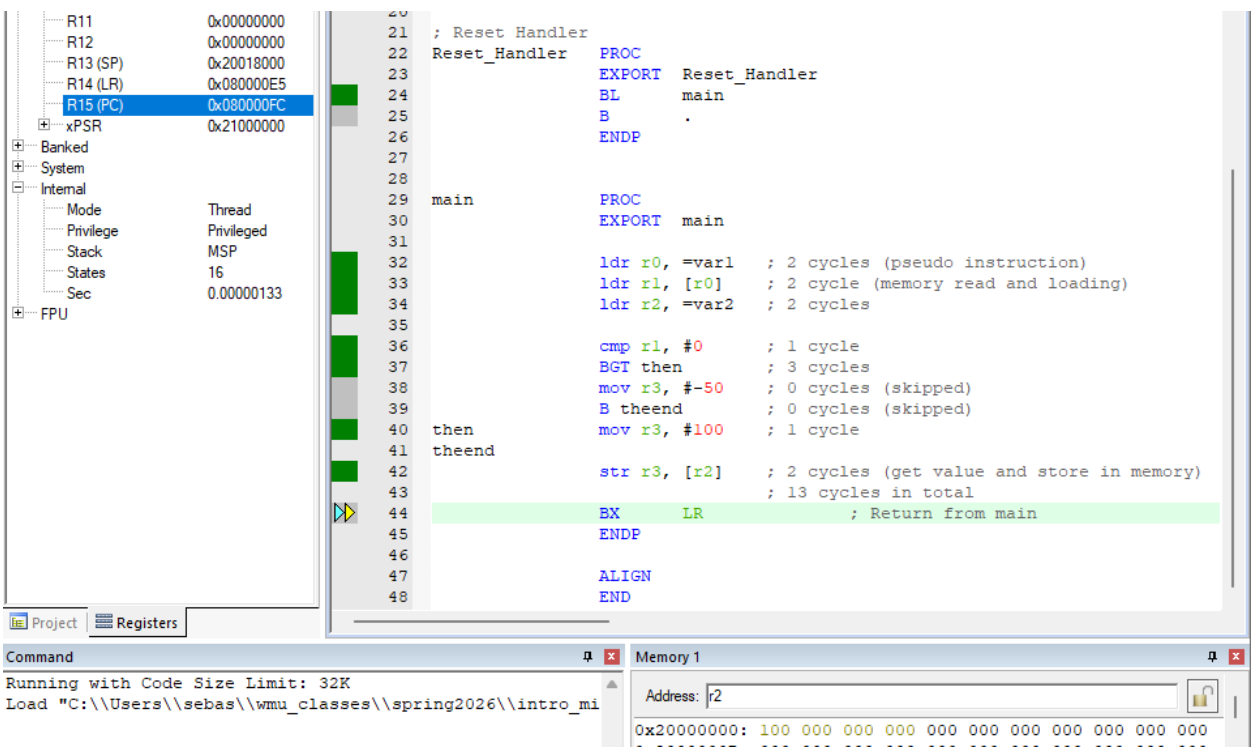


• Snapshot for the execution window showing the execution time.

Start:



End:

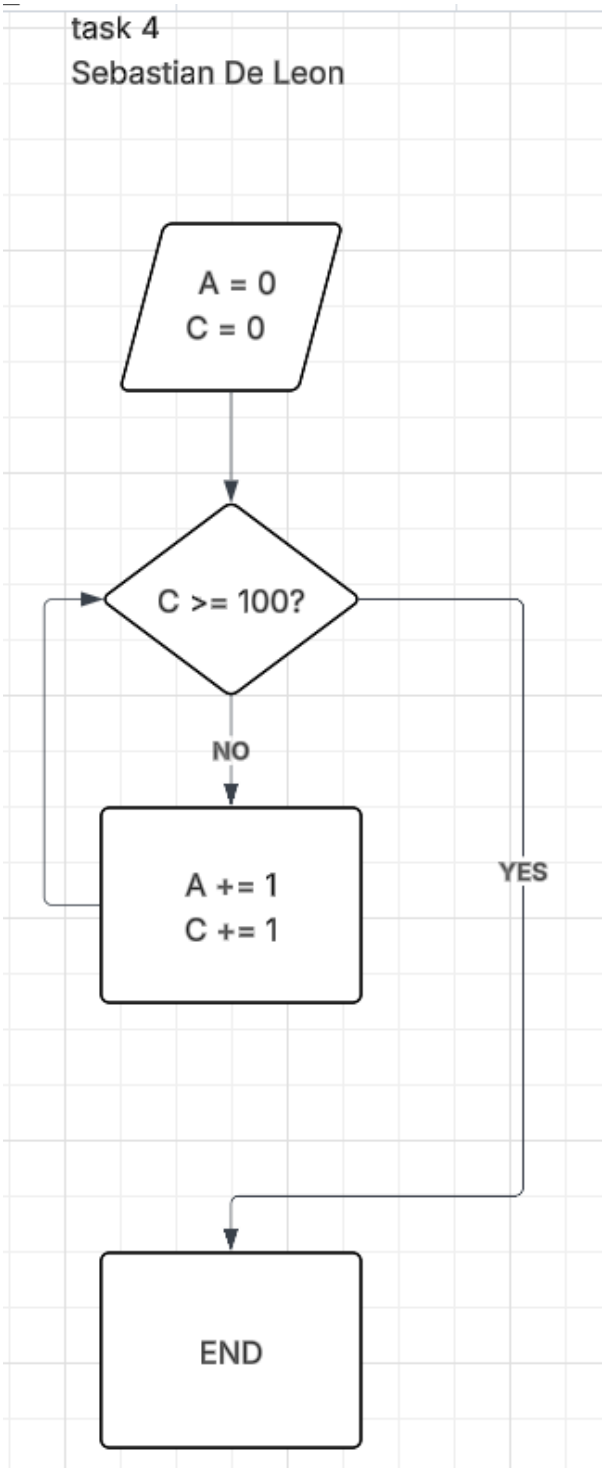


So, the total cycles by the program is 16 – 3 which is equal to 13 cycles as my calculations predicted.

(I did my hand calculations on the comments of my program as seen in the figures up)

Task 4:

• Draw a flowchart for your program.



• Write the assembly code.

C:\Users\sebas\wmu_classes\spring2026\intro_microprocessors\lab\lab5\task4\task4.uvprojx - µVision [Non-Commercial Use License]

File Edit View Project Flash Debug Peripherals Tools SVCS Window Help

Target_1

Project

- Project: task4
  - Target_1
    - Source Group 1
    - CMSIS

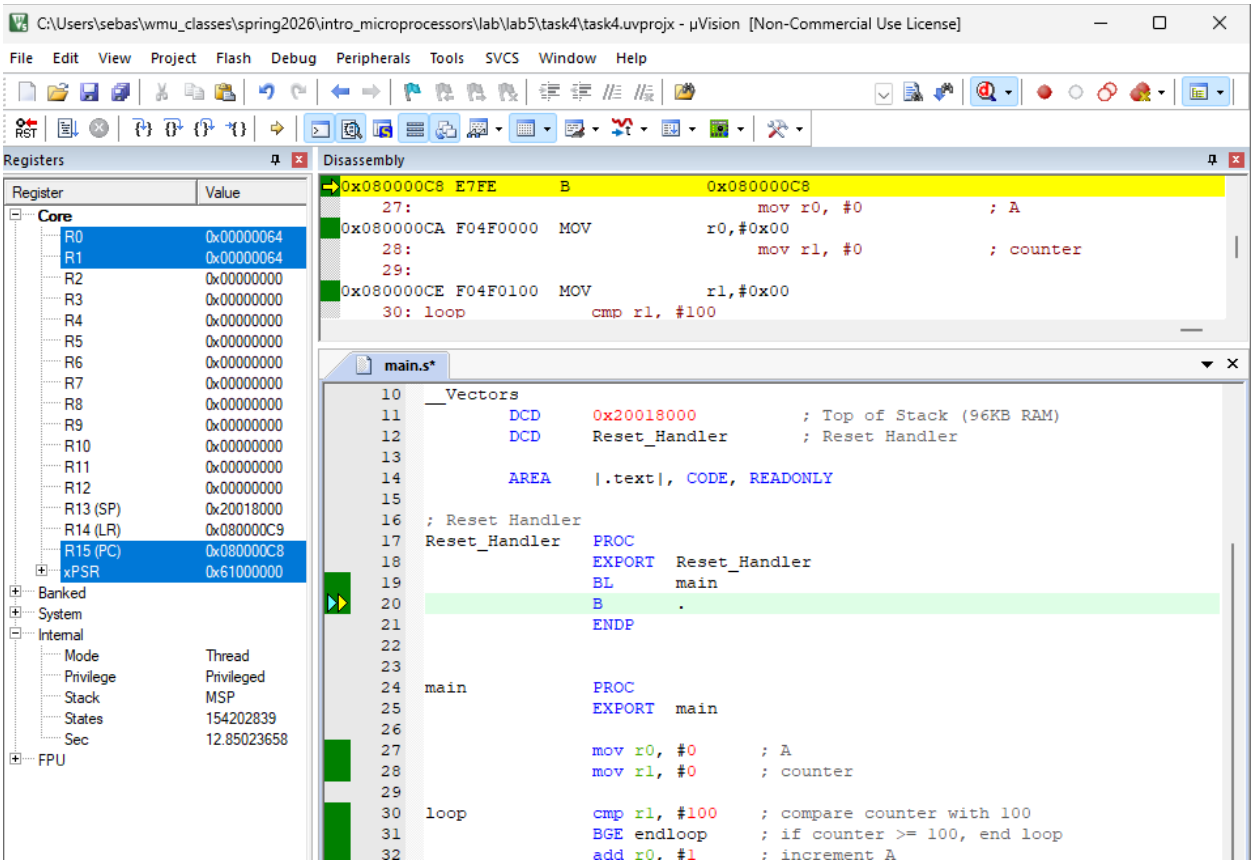
main.s*

```
;ARM assembly code shell

        PRESERVE8
        THUMB

; Vector Table
        AREA    RESET, DATA, READONLY
        EXPORT  __Vectors

__Vectors
        DCD     0x20018000          ; Top of Stack (96KB RAM)
        DCD     Reset_Handler       ; Reset Handler

        AREA    |.text|, CODE, READONLY

; Reset Handler
Reset_Handler   PROC
                EXPORT  Reset_Handler
                BL      main
                B       .
                ENDP


main            PROC
                EXPORT  main

                mov r0, #0      ; A
                mov r1, #0      ; counter

loop            cmp r1, #100    ; compare counter with 100
                BGE endloop     ; if counter >= 100, end loop
                add r0, #1      ; increment A
                add r1, #1      ; increment counter
                B loop          ; branch to loop
endloop

                BX      LR              ; Return from main
                ENDP

                ALIGN
                END
```

Build Output

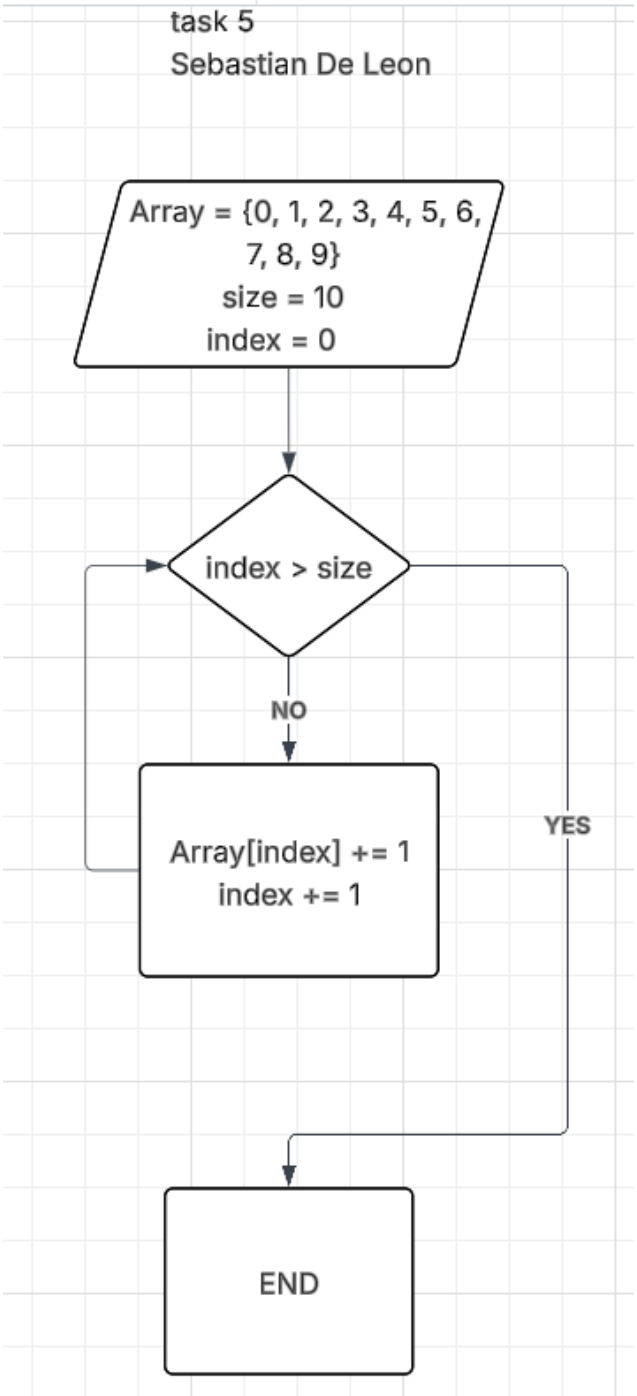
```
Build started: Project: task4
*** Using Compiler 'V6.24', folder: 'C:\Users\sebas\AppData\Local\Keil_v5\ARM\ARMCLANG\Bin'
Build target 'Target_1'
".\Objects\task4.axf" - 0 Error(s), 0 Warning(s).
Build Time Elapsed:  00:00:00
```

• Screenshot showing the results.



Task 5:

• Draw a flow chart for your program.



- Write the assembly program. Define all variables and the array.

C:\Users\sebas\wmu_classes\spring2026\intro_microprocessors\lab\lab5\task5\task5.uvprojx - µVision [Non-Commercial Use License]

File Edit View Project Flash Debug Peripherals Tools SVCS Window Help

Target_1

Project: task5
- Target_1
  - Source Group 1
  - CMSIS

main.s

```
;ARM assembly code shell

        PRESERVE8
        THUMB

; Vector Table
        AREA    RESET, DATA, READONLY
        EXPORT  __Vectors

__Vectors
        DCD     0x20018000          ; Top of Stack (96KB RAM)
        DCD     Reset_Handler       ; Reset Handler

        AREA    arrayData, DATA, READONLY
source_array    DCB     0x0, 0x1, 0x2, 0x3, 0x4, 0x5, 0x6, 0x7, 0x8, 0x9; declarin

        AREA    myData, DATA, READWRITE
dest_array      space   10
size            equ     10  ; declaring size of array
index           equ     0   ; declaring index

        AREA    |.text|, CODE, READONLY

; Reset Handler
Reset_Handler   PROC
                EXPORT  Reset_Handler
                BL      main
                B       .
                ENDP


main            PROC
                EXPORT  main

                ldr r0, =source_array   ; loading array address in r0
                ldr r1, =dest_array
                mov r2, #index          ; current index

copy_arr_loop
                cmp r2, #size
                BGE end_copy_arr_loop
                ldrb r3, [r0, r2]
                strb r3, [r1, r2]
                add r2, r2, #1
                B copy_arr_loop
end_copy_arr_loop

                mov r2, #index          ; reseting r2 index to 0

loop            cmp r2, #size           ; comparing index to the size

                BGE endloop             ; if index >= size end loop

                ldrb r3, [r1, r2]       ; load byte in index to r2

                add r3, r3, #1          ; add 1 to the byte in r2
```

C:\Users\sebas\wmu_classes\spring2026\intro_microprocessors\lab\lab5\task5\task5.uvprojx - µVision [Non-Commercial Use License]

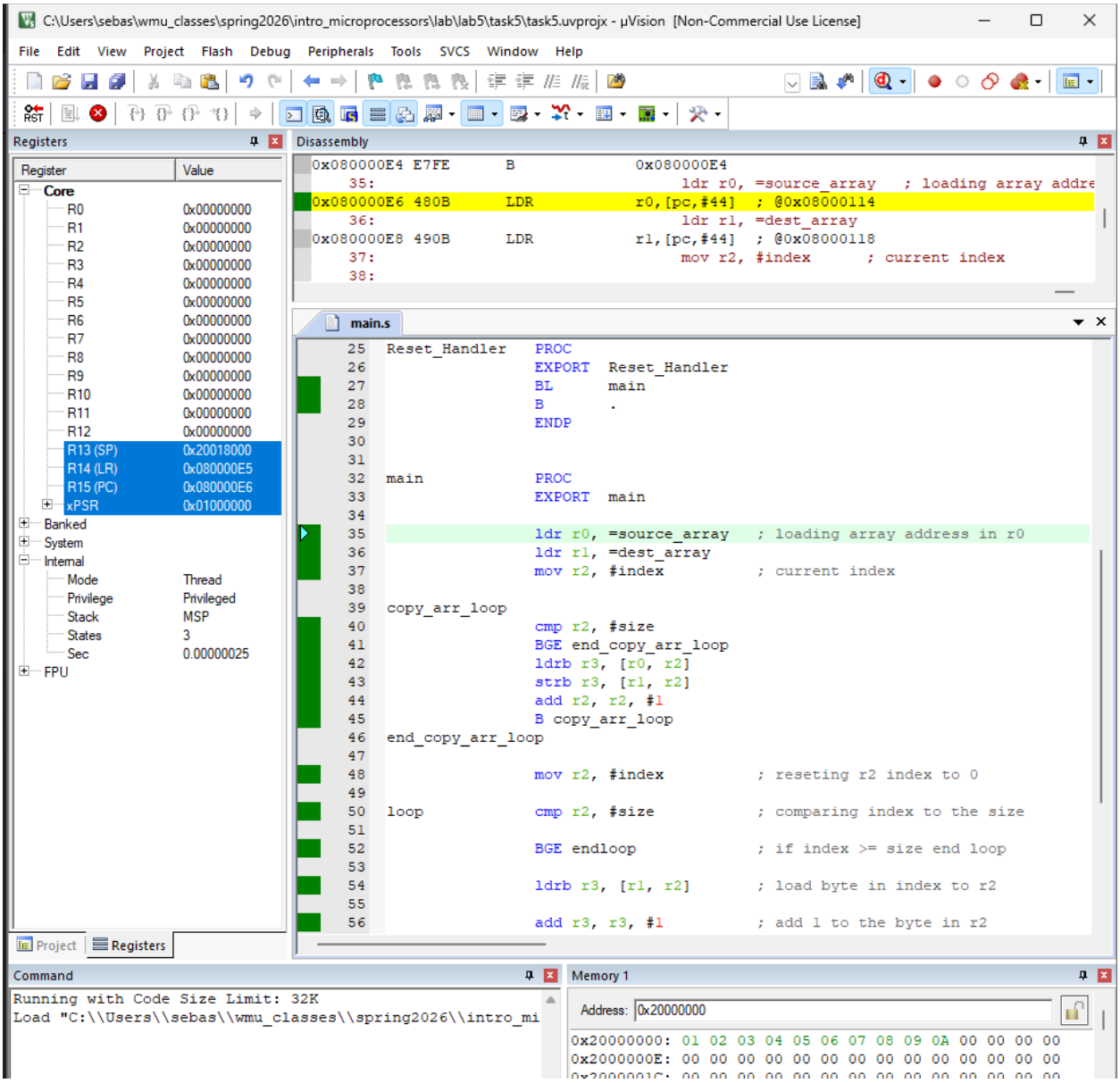
Project: task5
Target_1
Source Group 1
CMSIS

main.s

```
source_array    DCB     0x0, 0x1, 0x2, 0x3, 0x4, 0x5, 0x6, 0x7, 0x8, 0x9; declarin

        AREA    myData, DATA, READWRITE
dest_array      space   10
size            equ     10  ; declaring size of array
index           equ     0   ; declaring index

        AREA    |.text|, CODE, READONLY

; Reset Handler
Reset_Handler   PROC
                EXPORT  Reset_Handler
                BL      main
                B       .
                ENDP


main            PROC
                EXPORT  main

                ldr r0, =source_array   ; loading array address in r0
                ldr r1, =dest_array
                mov r2, #index          ; current index

copy_arr_loop
                cmp r2, #size
                BGE end_copy_arr_loop
                ldrb r3, [r0, r2]
                strb r3, [r1, r2]
                add r2, r2, #1
                B copy_arr_loop
end_copy_arr_loop

                mov r2, #index          ; reseting r2 index to 0

loop            cmp r2, #size           ; comparing index to the size

                BGE endloop             ; if index >= size end loop

                ldrb r3, [r1, r2]       ; load byte in index to r2

                add r3, r3, #1          ; add 1 to the byte in r2

                strb r3, [r1, r2]       ; store the byte in r2 back in the
                                        ; same position in the array

                add r2, r2, #1          ; add 1 to index

                B loop                  ; branch to loop
endloop

                BX      LR              ; Return from main
                ENDP

                ALIGN
                END
```

Simulation

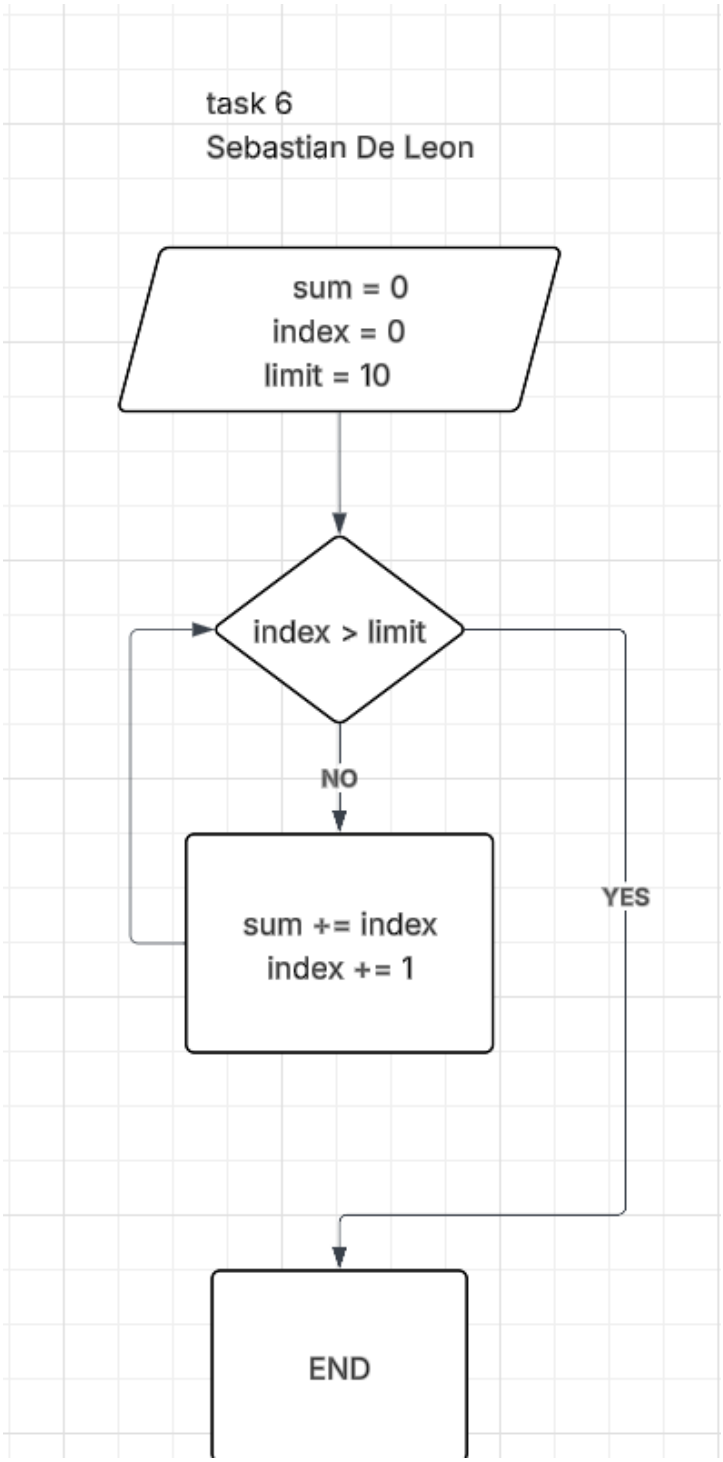
• Screenshot showing the results.



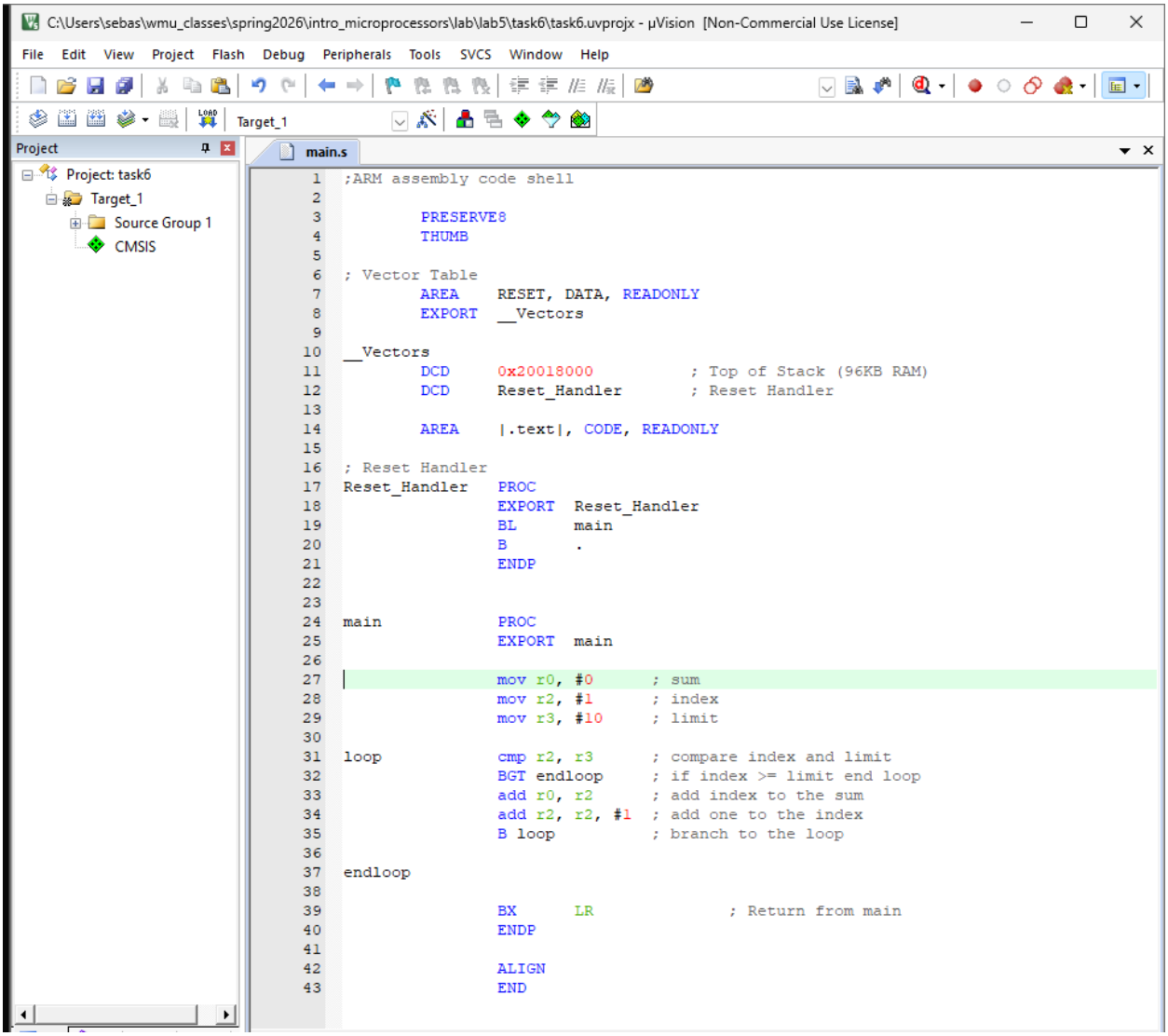
For task 5 I had a problem in which every time r2 gets loaded it was being loaded with zero, adding 1 and storing 1 so at the end I was getting an array of all ones.
It was because the values were being stored in flash memory so I had to clone the array into ram, or initialize the array inside the main code, but I was to do It this way to practice

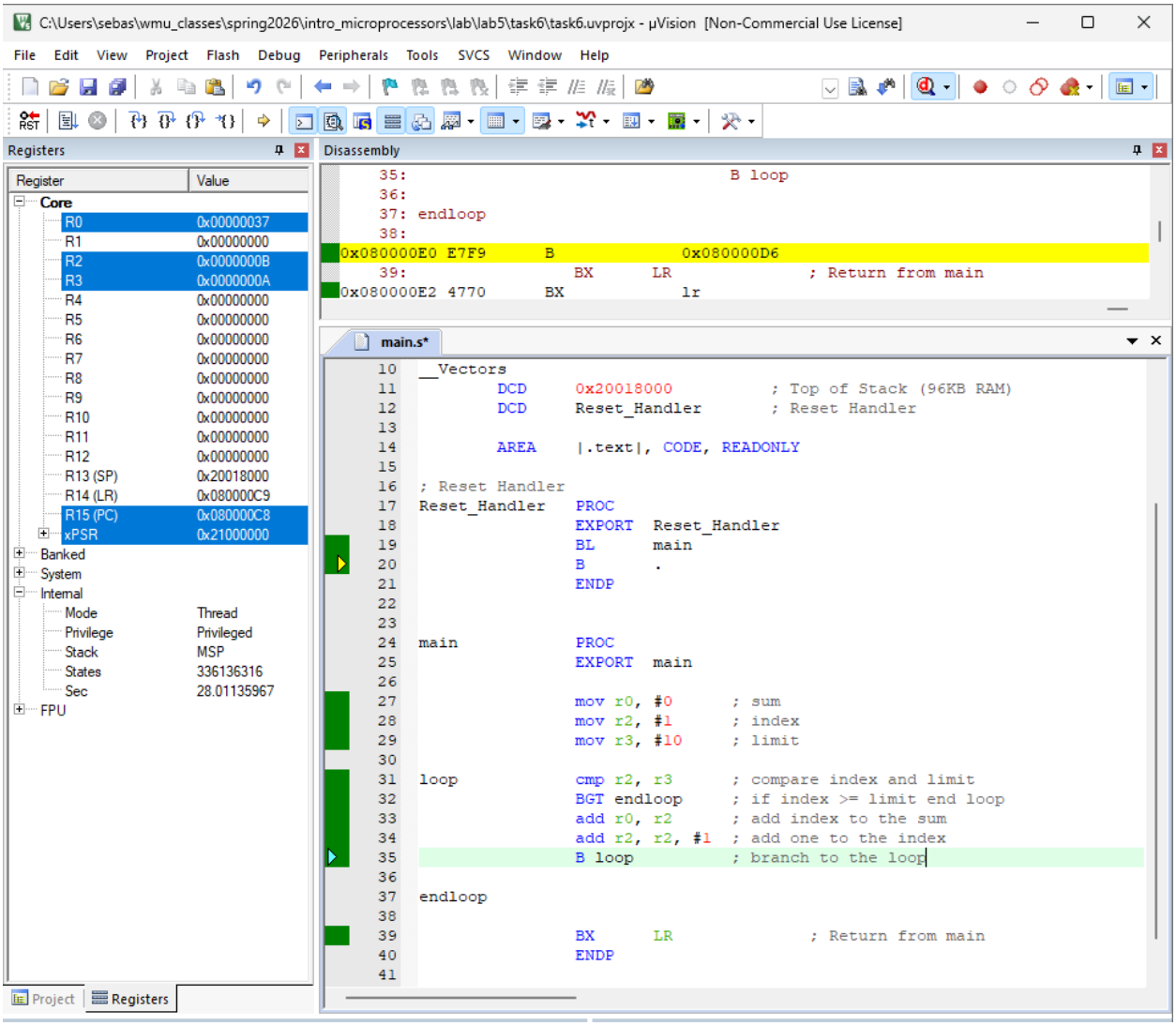
Task 6:

• Draw a flowchart of your program.



• Write the assembly code.




```
;ARM assembly code shell

        PRESERVE8
        THUMB

; Vector Table
        AREA    RESET, DATA, READONLY
        EXPORT  __Vectors

__Vectors
        DCD     0x20018000          ; Top of Stack (96KB RAM)
        DCD     Reset_Handler       ; Reset Handler

        AREA    |.text|, CODE, READONLY

; Reset Handler
Reset_Handler   PROC
                EXPORT  Reset_Handler
                BL      main
                B       .
                ENDP


main            PROC
                EXPORT  main

                mov r0, #0      ; sum
                mov r2, #1      ; index
                mov r3, #10     ; limit

loop            cmp r2, r3      ; compare index and limit
                BGT endloop     ; if index >= limit end loop
                add r0, r2      ; add index to the sum
                add r2, r2, #1  ; add one to the index
                B loop          ; branch to the loop

endloop

                BX      LR              ; Return from main
                ENDP

                ALIGN
                END
```

• Screenshot showing the results